DOCUMENT 11 — INGREDIENT & RECIPE DATA MANAGEMENT SPECIFICATION

1. Dokümanın Amacı

Bu dokümanın amacı Smart Chef & Pantry AI uygulamasında kullanılan ingredient ve recipe verilerinin nasıl oluşturulacağını, saklanacağını, normalize edileceğini, birbirleriyle ilişkilendirileceğini ve yönetileceğini tanımlamaktır.

Bu doküman özellikle aşağıdaki konuları standartlaştırır:

- Ingredient veri modeli
- Recipe veri modeli
- Recipe–Ingredient ilişkisi
- Ingredient normalization
- Optional / required ingredient ayrımı
- Quantity ve unit yönetimi
- Recipe source yönetimi
- YouTube URL yönetimi
- Admin tarafından veri yönetimi
- Veri doğrulama kuralları
- Duplicate ingredient kontrolü
- Aktif/pasif recipe ve ingredient yönetimi
- Seed data yaklaşımı
- API ve database arasındaki veri dönüşümü
- Veri bütünlüğü ve test yaklaşımı

Bu dokümanın temel amacı, recommendation sisteminin güvenilir ve tutarlı ingredient/recipe verileri üzerinden çalışmasını sağlamaktır.

---

2. Veri Yönetimi Genel Mimarisi

Ingredient ve recipe verileri aşağıdaki mimari üzerinden yönetilecektir:

Admin / Seed Data
↓
API / Data Import
↓

Validation
↓
Service Layer
↓
Repository Layer
↓
SQLAlchemy Models
↓
PostgreSQL
↓
Recipe Matching Engine
↓
Recommendation API
↓
Flutter

Flutter tarafı doğrudan database'e erişmeyecektir.

Flutter
↓
REST API
↓
Flask
↓
PostgreSQL

şeklinde katmanlı yapı korunacaktır.

---

3. Ingredient Kavramı

Ingredient, kullanıcının evinde bulunabilecek veya bir tarif içerisinde kullanılabilecek temel gıda/malzeme bilgisidir.

Örnek:

Domates
Soğan
Sarımsak
Yumurta
Un
Süt
Peynir
Tavuk

Pirinç
Makarna

Ingredient sistemi recommendation algoritmasının temel girdilerinden biridir.

Kullanıcının seçtiği:

Domates
Soğan
Yumurta

ingredient'ları recipe matching sistemine gönderilir.

---

4. Ingredient Database Model

Ingredient tablosu:

ingredients
---
id
name
normalized_name
category
created_at
updated_at

Alanların görevleri:

Alan	Açıklama
id	UUID primary key
name	Kullanıcıya gösterilecek ingredient adı
normalized_name	Matching ve duplicate kontrolünde kullanılacak normalize edilmiş ad
category	Ingredient kategorisi
created_at	Oluşturulma tarihi
updated_at	Güncellenme tarihi

---

5. Ingredient ID

Ingredient ID olarak UUID kullanılacaktır.

Örnek:

550e8400-e29b-41d4-a716-446655440000

UUID kullanılmasının nedenleri:

- Tahmin edilmesi zor ID
- Dağıtık sistemlere uygunluk
- Database büyüdüğünde esneklik
- API tarafında güvenli resource identification
- Integer ID'lere göre daha düşük enumeration riski

Flutter tarafında ID string olarak temsil edilecektir.

---

6. Ingredient Name

`name` alanı kullanıcıya gösterilecek gerçek isimdir.

Örnek:

name = "Domates"

Bu alan kullanıcı deneyimi için önemlidir.

Kullanıcı:

Domates

görmelidir.

Database'in internal matching işlemleri için ise `normalized_name` kullanılacaktır.

---

7. Ingredient Normalization

Ingredient normalization, ingredient isimlerinin matching için standart hale getirilmesi işlemidir.

Örneğin:

" Domates "
"DOMATES"
"domates"

gibi değerler normalize edildiğinde:

domates

haline getirilebilir.

Temel normalization işlemleri:

Trim whitespace
↓
Lowercase
↓
Normalize repeated spaces
↓
Normalize basic textual formatting

Örnek:

" Domates "
↓
"domates"

---

8. Semantic Ingredient Variants

Normalization sistemi semantic olarak farklı ingredient'ları otomatik olarak aynı kabul etmeyecektir.

Örneğin:

Domates
Çeri Domates
Domates Sosu

otomatik olarak:

domates

şeklinde birleştirilmeyecektir.

Bunun nedeni bu ingredient'ların gerçek kullanım açısından birbirinden farklı olabilmesidir.

Örneğin:

Recipe:
Çeri Domates

User:
Domates

durumunda sistem otomatik olarak %100 eşleşme varsaymamalıdır.

---

9. Ingredient Alias System

İlerleyen sürümlerde controlled alias sistemi eklenebilir.

Örneğin:

ingredient_aliases
---
id
canonical_ingredient_id
alias
normalized_alias

Örnek:

Canonical:
Zeytinyağı

Aliases:
Olive Oil
Olive-oil
Zeytin yağı

Ancak alias sistemi MVP'de zorunlu değildir.

MVP'de yalnızca güvenli ve kontrollü normalization uygulanacaktır.

---

10. Ingredient Category

Ingredient'lar category alanına sahip olacaktır.

Örnek kategoriler:

Vegetable
Fruit
Meat
Dairy
Grain
Legume
Spice
Sauce
Oil
Beverage
Other

Türkçe kullanıcı arayüzünde:

Sebze
Meyve
Et
Süt Ürünleri
Tahıl
Bakliyat
Baharat
Sos
Yağ
İçecek
Diğer

şeklinde gösterilebilir.

Backend tarafında category değerlerinin standart bir enum veya kontrollü string listesi üzerinden yönetilmesi önerilir.

---

11. Recipe Kavramı

Recipe, kullanıcının hazırlayabileceği yemek veya yiyecek tarifidir.

Recipe aşağıdaki temel bilgileri içerir:

Title
Description
Image
Instructions
Source URL
YouTube URL
Category
Ingredients

Recipe ve ingredient arasında doğrudan many-to-many ilişki vardır.

---

12. Recipe Database Model

`recipes` tablosu:

recipes
---
id
title
description
image_url
instructions
source_url
youtube_url
category
created_at
updated_at

Alanların görevleri:

Alan	Açıklama
id	Recipe UUID
title	Tarif adı
description	Tarif açıklaması
image_url	Tarif görseli

instructions	Hazırlama adımları
source_url	Orijinal tarif kaynağı
youtube_url	İlgili YouTube videosu
category	Tarif kategorisi
created_at	Oluşturulma tarihi
updated_at	Güncellenme tarihi

---

13. Recipe Title

Recipe title kullanıcıya gösterilecek tarif ismidir.

Örnek:

Menemen
Tavuklu Makarna
Mercimek Çorbası
Sebzeli Omlet

Title boş bırakılamaz.

Minimum doğrulama:

title != null
title != ""

Ayrıca gereksiz whitespace temizlenmelidir.

---

14. Recipe Description

Description tarif hakkında kısa açıklamadır.

Örnek:

Domates, biber ve yumurta ile hazırlanan pratik bir kahvaltı tarifi.

Description:

- nullable olabilir
- kullanıcıya recipe detail ekranında gösterilebilir
- recommendation algoritmasının temel matching girdisi değildir

---

15. Recipe Image

Recipe görseli:

image_url

alanında URL olarak tutulacaktır.

MVP'de görseller database binary data olarak saklanmayacaktır.

Yanlış yaklaşım:

Database
↓
BLOB image

Tercih edilen:

Database
↓
image_url
↓
External / Managed Image Storage

Flutter:

image_url
↓
CachedNetworkImage

kullanarak görseli yükleyebilir.

---

16. Image URL Validation

Image URL varsa geçerli bir URL formatında olması beklenir.

Örnek:

https://example.com/images/menemen.jpg

Geçersiz:

not-a-url

Ancak görsel URL'si MVP için nullable olabilir.

Görsel bulunmayan tariflerde Flutter tarafında placeholder kullanılmalıdır.

---

17. Recipe Instructions

Recipe instructions, tarifin hazırlanış adımlarını içerir.

Örneğin:

1. Soğanları doğrayın.
2. Tavayı ısıtın.
3. Domatesleri ekleyin.
4. Yumurtaları kırın.
5. Pişene kadar karıştırın.

MVP'de instructions tek bir text alanında saklanabilir.

Gelecekte daha gelişmiş step-based yapı oluşturulabilir:

recipe_steps
---
id
recipe_id
step_number
instruction

Ancak MVP için ayrı tablo zorunlu değildir.

---

18. Recipe Source URL

Her tarifin mümkünse orijinal kaynağı tutulacaktır.

Alan:

source_url

Örnek:

https://example.com/recipes/menemen

Kullanıcı Recipe Detail ekranında:

Tarifin Kaynağını Aç

butonuna bastığında bu URL Flutter tarafından açılacaktır.

---

19. YouTube URL

Tarife ait YouTube videosu varsa:

youtube_url

alanında tutulacaktır.

Örnek:

https://www.youtube.com/watch?v=...

Bu alan nullable olacaktır.

Yani her tarifin YouTube videosu olması zorunlu değildir.

---

20. Recipe Category

Recipe category, tarifin genel kategorisini belirtir.

Örnek:

Breakfast
Main Course
Soup
Dessert
Salad
Snack
Pasta
Vegetarian

Türkçe UI:

Kahvaltı
Ana Yemek
Çorba
Tatlı
Salata
Atıştırmalık
Makarna
Vejetaryen

Category değerlerinin ilerleyen sürümlerde ayrı bir `recipe_categories` tablosuna taşınması mümkündür.

MVP'de kontrollü string/enum yaklaşımı yeterlidir.

---

21. Recipe–Ingredient Relationship

Bir recipe birden fazla ingredient içerebilir.

Bir ingredient da birden fazla recipe içerisinde kullanılabilir.

Bu nedenle ilişki:

Recipe
↕
Many-to-Many
↕
Ingredient

şeklindedir.

Arada:

recipe_ingredients

tablosu bulunacaktır.

---

22. Recipe Ingredients Table

Tablo:

recipe_ingredients
---
id
recipe_id
ingredient_id
quantity
unit
display_quantity
is_optional

Alanlar:

Alan	Açıklama
id	UUID
recipe_id	Recipe foreign key
ingredient_id	Ingredient foreign key
quantity	Sayısal miktar
unit	Ölçü birimi
display_quantity	Kullanıcıya gösterilecek miktar
is_optional	Ingredient opsiyonel mi?

---

23. Quantity

Quantity ingredient miktarını temsil eder.

Örnek:

2
1
0.5
3

Database'de `DECIMAL` kullanılması önerilir.

Örneğin:

2.5
0.5
1.25

gibi değerlerin doğru şekilde saklanması gerekir.

Floating point rounding problemlerini azaltmak amacıyla quantity için uygun precision/scale belirlenmelidir.

---

24. Unit

Unit ingredient ölçü birimidir.

Örnek:

adet
gram
kg
ml
litre
çay kaşığı
yemek kaşığı
su bardağı

Örnek recipe ingredient:

Ingredient:
Domates

Quantity:
3

Unit:
adet

Kullanıcıya:

3 adet domates

olarak gösterilebilir.

---

25. Display Quantity

`display_quantity`, kullanıcıya gösterilecek quantity bilgisini desteklemek için kullanılabilir.

Örneğin database'de:

quantity = 1.5
unit = "cup"

olmasına rağmen kullanıcıya:

1,5 su bardağı

gösterilebilir.

Bu alan özellikle farklı dil, locale ve kullanıcı gösterim ihtiyaçlarında faydalıdır.

MVP'de quantity ve unit yeterli görülebilir; ancak display_quantity alanı recipe verisinin presentation detaylarını korumak için desteklenebilir.

---

26. Required vs Optional Ingredients

Recipe ingredient'larının iki türü olacaktır:

Required
Optional

Database:

is_optional = false

→ Required

is_optional = true

→ Optional

Örnek:

Menemen

Domates Required
Yumurta Required
Biber Required
Tuz Optional
Maydanoz Optional

Optional ingredient'lar recipe matching percentage hesabında kullanıcıyı cezalandırmayacaktır.

---

27. Recipe Matching ile İlişki

Recipe Matching algoritması:

Required Ingredients
↓
User Ingredients
↓
Matched / Missing
↓
Match Percentage

hesaplar.

Örneğin:

Recipe:

Domates
Yumurta
Biber
Tuz

Bunlardan:

Domates → Required
Yumurta → Required
Biber → Required
Tuz → Optional

kabul edilirse ve kullanıcı:

Domates
Yumurta

seçerse:

Matched Required = 2
Total Required = 3

olur.

Match percentage:

2 / 3 × 100 = 66.67%

olacaktır.

Tuzun eksik olması percentage'i düşürmez.

---

28. Duplicate Ingredient Rule

Aynı recipe içerisinde aynı ingredient normal şartlarda iki ayrı kayıt olarak bulunmamalıdır.

Örneğin:

Recipe:
Menemen

Ingredient:
Domates

Ingredient:

Domates

şeklinde duplicate oluşturulmamalıdır.

Database seviyesinde uygun bir unique constraint uygulanmalıdır:

recipe_id + ingredient_id

unique olmalıdır.

---

29. Duplicate Ingredient Selection

Kullanıcı aynı ingredient'ı birden fazla kez seçse bile matching hesabında ingredient yalnızca bir kez sayılmalıdır.

Örneğin:

User selection:
Domates
Domates
Yumurta

sistemde:

Domates
Yumurta

olarak değerlendirilmelidir.

Bu kontrol recommendation service / matcher katmanında yapılacaktır.

---

30. Ingredient Deletion

Ingredient silme işlemi dikkatli yapılmalıdır.

Bir ingredient:

recipe_ingredients

tablosunda kullanılıyorsa doğrudan silinmesi recipe verilerinin bütünlüğünü etkileyebilir.

Bu nedenle MVP'de aktif/pasif yaklaşımı tercih edilmesi önerilir.

Örneğin:

is_active

alanı ilerleyen schema versiyonuna eklenebilir.

Alternatif olarak ingredient silme işlemi kullanım durumuna göre sınırlandırılabilir.

---

31. Recipe Deletion

Recipe için de fiziksel silme yerine pasifleştirme tercih edilebilir.

Önerilen yaklaşım:

Recipe Active
↓
is_active = true

Pasifleştirme:

is_active = false

Recommendation sistemi:

WHERE is_active = true

olan tarifleri kullanır.

Bu yaklaşım:

- Eski verilerin kaybolmasını önler
- Favorite kayıtlarının bozulmasını azaltır
- Yanlışlıkla silinen recipe'nin geri alınmasını kolaylaştırır
- Audit ihtiyacını ileride destekler

---

32. Active Ingredient / Recipe

Recipe modeline:

is_active

ve ingredient modeline de gerekirse:

is_active

alanları eklenebilir.

MVP database tasarımında bu alanlar ilk versiyonda bulunmuyorsa migration ile sonradan eklenebilir.

Önerilen mantık:

Active Recipe
↓
Recommendation'a dahil edilir

Inactive Recipe
↓
Recommendation'a dahil edilmez

Ingredient için:

Inactive Ingredient
↓
Yeni recipe verisine eklenemez

Ancak geçmiş recipe ilişkileri veri bütünlüğünü koruyacak şekilde saklanabilir.

---

33. Recipe Data Validation

Recipe oluşturulurken aşağıdaki alanlar kontrol edilmelidir:

title
description
image_url
instructions

source_url
youtube_url
category
ingredients

Minimum zorunlu alanlar:

title
ingredients
instructions

Recipe için en az bir required ingredient bulunmalıdır.

---

34. Recipe Validation Rules

Recipe oluşturulurken:

title boş olamaz

ingredients boş olamaz

en az bir required ingredient bulunmalıdır

ingredient_id geçerli olmalıdır

quantity negatif olamaz

URL alanları geçerli formatta olmalıdır

aynı ingredient aynı recipe içerisinde tekrar edemez

kontrolleri uygulanacaktır.

---

35. Ingredient Validation Rules

Ingredient oluşturulurken:

name boş olamaz

normalized_name otomatik oluşturulmalıdır

normalized_name duplicate olmamalıdır

category geçerli olmalıdır

kontrolleri yapılacaktır.

Örneğin:

"Domates"

oluşturulurken:

normalized_name = "domates"

otomatik hesaplanmalıdır.

Admin'in normalized_name değerini manuel girmesi gerekmemelidir.

---

36. Admin Recipe Creation

Admin recipe oluşturmak için:

POST /api/admin/recipes

endpoint'ini kullanacaktır.

Örnek request:

```
{
  "title": "Menemen",
  "description": "Pratik bir kahvaltı tarifi.",
  "image_url": "https://example.com/menemen.jpg",
  "instructions": "Soğanları doğrayın...",
  "source_url": "https://example.com/menemen",
  "youtube_url": "https://youtube.com/watch?v=example",
  "category": "breakfast",
  "ingredients": [
    {
      "ingredient_id": "ingredient-001",
      "quantity": 3,
```

```
      "unit": "adet",
      "is_optional": false
    },
    {
      "ingredient_id": "ingredient-002",
      "quantity": 2,
      "unit": "adet",
      "is_optional": false
    },
    {
      "ingredient_id": "ingredient-003",
      "quantity": 1,
      "unit": "çay kaşığı",
      "is_optional": true
    }
  ]
}
```

Backend:

Request
↓
Schema Validation
↓
Ingredient Validation
↓
Duplicate Check
↓
Recipe Creation
↓
Recipe Ingredient Creation
↓
Transaction Commit

şeklinde çalışacaktır.

---

37. Recipe Creation Transaction

Recipe ve recipe_ingredients birbiriyle bağlantılı olduğundan creation işlemi transaction içerisinde yapılmalıdır.

Örneğin:

Begin Transaction
↓
Create Recipe
↓
Create Recipe Ingredients
↓
Validation
↓
Commit

Herhangi bir işlem başarısız olursa:

Rollback

yapılmalıdır.

Böylece recipe oluşturulup ingredient ilişkileri oluşturulamadan sistemde yarım recipe kalması önlenir.

---

38. Recipe Update

Admin:

PUT /api/admin/recipes/{recipe_id}

ile recipe güncelleyebilir.

Güncellenebilecek alanlar:

title
description
image_url
instructions
source_url
youtube_url
category
ingredients
is_active

Recipe ingredient listesi değiştirilecekse ilgili transaction içerisinde mevcut ilişkiler kontrollü şekilde güncellenmelidir.

---

39. Recipe Update Strategy

Önerilen yaklaşım:

Load Recipe
↓
Validate Request
↓
Update Recipe Fields
↓
Validate Ingredient List
↓
Synchronize Recipe Ingredients
↓
Commit

Ingredient synchronization sırasında:

Existing
New
Removed
Updated

kayıtlar karşılaştırılabilir.

MVP'de daha basit bir yaklaşım olarak mevcut `recipe_ingredients` kayıtlarının kontrollü şekilde silinip yeni listenin transaction içerisinde oluşturulması da uygulanabilir.

Ancak bu işlem foreign key ve favorite ilişkileriyle karıştırılmamalıdır.

---

40. Recipe Detail API Data

Flutter'ın recipe detail ekranı için backend aşağıdaki yapıya benzer veri döndürebilir:

```
{
  "success": true,
  "data": {
    "id": "recipe-001",
    "title": "Menemen",
    "description": "Pratik bir kahvaltı tarifi.",
```

```
    "image_url": "https://example.com/menemen.jpg",
    "instructions": "Soğanları doğrayın...",
    "source_url": "https://example.com/menemen",
    "youtube_url": "https://youtube.com/watch?v=example",
    "category": "breakfast",
    "ingredients": [
      {
        "ingredient_id": "ingredient-001",
        "name": "Domates",
        "quantity": 3,
        "unit": "adet",
        "is_optional": false
      },
      {
        "ingredient_id": "ingredient-002",
        "name": "Yumurta",
        "quantity": 2,
        "unit": "adet",
        "is_optional": false
      }
    ]
  }
}
```


Flutter bu response'u API modeline dönüştürecektir.

---

41. Recommendation API Data

Recommendation endpoint'inde recipe detail'den farklı olarak matching sonucu da gönderilecektir.

Örnek:


```
{
  "success": true,
  "data": {
    "items": [
      {
        "recipe_id": "recipe-001",
        "title": "Menemen",
        "image_url": "https://example.com/menemen.jpg",
        "match_percentage": 66.67,
        "matched_ingredients": [
          {
```

```
          "ingredient_id": "ingredient-001",
          "name": "Domates"
        },
        {
          "ingredient_id": "ingredient-002",
          "name": "Yumurta"
        }
      ],
      "missing_ingredients": [
        {
          "ingredient_id": "ingredient-003",
          "name": "Biber"
        }
      ]
    }
  ]
}
```

`match_percentage` backend tarafından hesaplanacaktır.

Flutter bu değeri yeniden hesaplamayacaktır.

---

42. Seed Data

MVP geliştirme sürecinde başlangıç recipe ve ingredient verilerinin hızlı oluşturulabilmesi için seed data kullanılabilir.

Örnek:

```
seed/
├── ingredients.json
└── recipes.json
```

veya Python tabanlı seed script:

```
scripts/
└── seed_database.py
```

kullanılabilir.

Seed işlemi:

Create Ingredients
↓
Create Recipes
↓
Create Recipe Ingredients
↓
Validate
↓
Commit

şeklinde çalışmalıdır.

---

43. Seed Data Kuralları

Seed data içerisinde:

- duplicate ingredient bulunmamalıdır
- geçersiz UUID kullanılmamalıdır
- olmayan ingredient ID'si recipe'ye bağlanmamalıdır
- negatif quantity kullanılmamalıdır
- recipe'de required ingredient bulunmalıdır
- URL formatları geçerli olmalıdır
- ingredient isimleri normalize edilebilir olmalıdır

Seed data production database'e doğrudan kontrolsüz şekilde yüklenmemelidir.

---

44. Recipe Data Source

Recipe verilerinin kaynağı proje kapsamında açıkça belirlenmelidir.

Kaynaklar:

Admin-created recipes
Authorized dataset
Manually prepared seed data
Approved external recipe data

kullanılabilir.

Recipe source URL varsa kullanıcıya kaynak gösterilebilir.

Telif veya kullanım koşulları açısından izin verilen veri kaynakları kullanılmalıdır.

---

45. External Recipe Content

Smart Chef & Pantry AI'nin MVP'si tariflerin tamamını başka sitelerden kontrolsüz şekilde kopyalamayı amaçlamaz.

Uygulamada:

Recipe metadata
+
Ingredient data
+
Instructions
+
Source URL

uygun şekilde yönetilmelidir.

External website'ten veri alınacaksa ilgili kaynağın:

- kullanım şartları
- API şartları
- veri lisansı
- scraping izinleri

kontrol edilmelidir.

---

46. Ingredient Search

Flutter'daki ingredient selection ekranında kullanıcı ingredient arayabilecektir.

API:

GET /api/ingredients?search=dom

örneği kullanılabilir.

Backend:

search

↓
normalized search
↓
database query
↓
pagination
↓
response

şeklinde çalışacaktır.

Örneğin:

dom

arama sonucu:

Domates
Domates Salçası

gibi kayıtlar dönebilir.

Semantic eşleştirme MVP'de yapılmayacaktır.

---

47. Ingredient Pagination

Ingredient sayısı arttığında tüm ingredient'ların tek response içerisinde gönderilmesi önerilmez.

Örneğin:

GET /api/ingredients?page=1&limit=20

response:

```
{
  "success": true,
  "data": {
    "items": [],
    "pagination": {
      "page": 1,
      "limit": 20,
```

```
    "total": 100,
    "total_pages": 5
  }
 }
}
```

şeklinde olabilir.

Bu yapı Flutter'ın büyük veri setlerinde daha verimli çalışmasını sağlar.

---

48. Ingredient Selection Data Flow

Kullanıcı ingredient seçtiğinde:

```
Ingredient Selection Screen
        ↓
Selected Ingredient IDs
        ↓
POST /api/recipes/recommend
        ↓
Recommendation Engine
        ↓
Recipe Matching
        ↓
Sorted Results
        ↓
Recipe Results Screen
```

Flutter yalnızca ingredient ID'lerini backend'e gönderebilir.

Örnek:

```
{
 "ingredient_ids": [
   "ingredient-001",
   "ingredient-002",
   "ingredient-003"
 ]
}
```

Ingredient isimlerinin tekrar gönderilmesi zorunlu değildir.

---

49. Ingredient Ownership

Ingredient'lar kullanıcıya özel değildir.

Yani:

User A
↓
Domates

User B
↓
Domates

aynı global ingredient kaydını kullanabilir.

Bu nedenle `ingredients` tablosunda `user_id` bulunmayacaktır.

Ingredient sistemi global katalog olarak çalışacaktır.

---

50. Recipe Ownership

MVP'de recipe'ler de global sistem verisidir.

Normal kullanıcı recipe oluşturmayacaktır.

Recipe oluşturma ve güncelleme:

Admin

yetkisine sahip olacaktır.

Bu nedenle:

recipes.user_id

alanı MVP'de gerekli değildir.

İleride kullanıcı tarafından oluşturulan tarifler eklenirse ayrı ownership modeli değerlendirilebilir.

---

51. Favorites ile Relationship

Favorite tablosu:

favorites
---
id
user_id
recipe_id
created_at

Recipe silinmesi/pasifleştirilmesi durumunda favorite ilişkilerinin nasıl yönetileceği belirlenmelidir.

Önerilen yaklaşım:

Recipe inactive
↓
Recipe recommendation'dan çıkar
↓
Existing favorite record korunabilir

Recipe detail endpoint'inde inactive recipe erişiminin davranışı ayrıca tanımlanmalıdır.

---

52. Cart ile Ingredient Relationship

Cart item:

cart_items
---
id
user_id
ingredient_id
quantity
unit
created_at
updated_at

şeklindedir.

Recipe'deki missing ingredients:

Missing Ingredient
↓
Add to Cart
↓
POST /api/cart/items

ile kullanıcı cart'ına eklenebilir.

---

53. Missing Ingredient → Cart Flow

Örneğin recipe:

Domates
Yumurta
Biber

Kullanıcı:

Domates
Yumurta

sahipse:

Missing:
Biber

olur.

Kullanıcı:

Sepete Ekle

dediğinde:

Biber
↓
Cart Item
↓
User Cart

oluşturulur.

---

54. Cart Merge Rule

Aynı ingredient sepete tekrar eklenirse duplicate cart item oluşturulmayacaktır.

Örneğin:

Cart:
Domates × 2

ve kullanıcı tekrar:

Domates × 3

eklerse:

Domates × 5

olmalıdır.

Unit farklıysa sistem bunu ayrı cart item olarak değerlendirebilir.

Örneğin:

Domates × 2 adet
Domates × 500 gram

aynı quantity olarak otomatik birleştirilmemelidir.

---

55. Data Integrity

Database foreign key ilişkileri korunmalıdır.

Örneğin:

recipe_ingredients.recipe_id
↓
recipes.id

ve:

recipe_ingredients.ingredient_id
↓
ingredients.id

olmalıdır.

Invalid foreign key oluşturulmasına izin verilmemelidir.

---

56. Recommended Database Indexes

Ingredient:

INDEX normalized_name
INDEX category

Recipe:

INDEX category
INDEX is_active

Recipe Ingredients:

INDEX recipe_id
INDEX ingredient_id
UNIQUE(recipe_id, ingredient_id)

Bu index'ler recommendation sorgularının performansına katkı sağlayacaktır.

---

57. Data Consistency Rules

Sistem aşağıdaki kuralları korumalıdır:

Bir recipe olmayan ingredient ile ilişkilendirilemez.

Bir ingredient olmayan recipe ile ilişkilendirilemez.

Aynı recipe içerisinde aynı ingredient duplicate olamaz.

Required ingredient sayısı 0 olan recipe recommendation sistemine dahil edilmez.

Negatif quantity kabul edilmez.

Ingredient normalized_name otomatik hesaplanır.

Flutter match_percentage hesaplamaz.

Backend recommendation sonucunu oluşturur.

---

58. Data Layer Responsibilities

Backend katmanları:

Route
↓
Schema
↓
Service
↓
Repository
↓
Model
↓
Database

şeklinde çalışacaktır.

Örneğin recipe creation:

admin_routes.py
↓
RecipeCreateSchema
↓
RecipeService.create_recipe()
↓
RecipeRepository
↓
SQLAlchemy
↓
PostgreSQL

Matching logic ise:

RecipeService
↓
RecommendationEngine
↓
RecipeMatcher
↓
Result

şeklinde çalışacaktır.

---

59. Repository Responsibility

Repository yalnızca database erişiminden sorumludur.

Örneğin:

RecipeRepository

şunları yapabilir:

get_by_id()
create()
update()
delete()
list_active()
get_with_ingredients()

Ancak:

match_percentage hesaplama

veya:

recommendation ranking

repository içerisinde yapılmamalıdır.

---

60. Service Responsibility

`RecipeService`:

- request doğrulamasının service seviyesindeki kontrollerini
- recipe business logic
- ingredient ilişkilerinin yönetimini
- recommendation engine çağrısını
- transaction yönetimini

koordine eder.

Ancak tüm matching algoritmasını kendi içerisinde tekrar yazmamalıdır.

Örneğin:

recommendation_engine.recommend(...)

çağrısı yapılabilir.

---

61. API ve Database Model Ayrımı

Database model doğrudan API response olarak döndürülmemelidir.

Yanlış:

SQLAlchemy Model
↓
JSON

Doğru:

SQLAlchemy Model
↓
Schema / Mapper
↓
API Model
↓
JSON

Böylece database yapısı ile public API contract birbirinden bağımsız tutulur.

---

62. Recipe Data Lifecycle

Recipe yaşam döngüsü:

Draft / Prepared
↓
Validated
↓
Active
↓
Updated
↓
Inactive

MVP'de draft workflow zorunlu değildir.

Basit durumda:

Create
↓
Validate
↓
Active

şeklinde olabilir.

---

63. Ingredient Data Lifecycle

Ingredient:

Create
↓
Normalize
↓
Duplicate Check
↓
Active

şeklinde ilerler.

Ingredient artık kullanılmayacaksa fiziksel silme yerine inactive yaklaşımı tercih edilebilir.

---

64. Admin Ingredient Management

Admin endpoint'leri:

POST   /api/admin/ingredients
PUT   /api/admin/ingredients/{id}
DELETE /api/admin/ingredients/{id}

Admin ingredient oluştururken:

```
{
  "name": "Domates",
  "category": "vegetable"
}
```

gönderebilir.

Backend:

name
↓
normalize
↓
duplicate check
↓
create

işlemini gerçekleştirir.

---

65. Admin Recipe Management

Admin endpoint'leri:

POST   /api/admin/recipes
PUT   /api/admin/recipes/{id}

DELETE /api/admin/recipes/{id}

Admin recipe yönetirken recipe ile ingredient ilişkilerini de yönetebilir.

Örneğin:

Menemen
├── Domates
├── Yumurta
├── Biber
└── Tuz

şeklindeki yapı tek bir recipe yönetim operasyonunun parçası olabilir.

---

66. Data Import Future Architecture

Recipe sayısı ileride manuel giriş için fazla hale gelirse import sistemi geliştirilebilir.

Örneğin:

CSV
JSON
External API
Authorized Dataset

kaynaklarından veri alınabilir.

Import pipeline:

Raw Data
↓
Parsing
↓
Normalization
↓
Validation
↓
Duplicate Detection
↓
Ingredient Mapping
↓
Recipe Creation
↓

Validation Report

şeklinde tasarlanmalıdır.

---

67. Import Duplicate Handling

Import sırasında:

"Tomato"
"tomato"
" TOMATO "

gibi değerler normalize edilerek duplicate kontrolünden geçirilmelidir.

Ancak:

Tomato
Cherry Tomato
Tomato Sauce

semantic olarak otomatik birleştirilmemelidir.

Bu eşleştirme gelecekte kontrollü alias sistemi üzerinden yapılabilir.

---

68. Recipe Quality Rules

Recommendation sisteminin kaliteli sonuç verebilmesi için recipe verileri:

- eksiksiz ingredient listesine sahip
- doğru ingredient ID'leri kullanan
- required / optional ayrımı doğru yapılmış
- anlaşılır instructions içeren
- mümkünse geçerli görsel URL'sine sahip
- mümkünse source URL'sine sahip

olmalıdır.

Özellikle ingredient listesi recommendation sisteminin doğrudan girdisi olduğu için veri kalitesi kritik öneme sahiptir.

---

69. Data Quality Check

Admin veya import işlemi sonrasında aşağıdaki kontroller yapılabilir:

Recipe has title?
Recipe has instructions?
Recipe has ingredients?
Recipe has required ingredient?
Ingredient IDs valid?
Duplicate ingredients?
Negative quantities?
Invalid URLs?
Missing category?
Inactive ingredient?

Hatalı kayıtların active recommendation sistemine girmesi engellenmelidir.

---

70. Test Cases

Ingredient Tests

Create valid ingredient
Create duplicate ingredient
Normalize uppercase ingredient
Normalize whitespace
Invalid empty ingredient
Invalid category

Recipe Tests

Create valid recipe
Create recipe without title
Create recipe without ingredients
Create recipe without required ingredient
Invalid ingredient ID
Duplicate recipe ingredient
Negative quantity
Invalid URL

Relationship Tests

Recipe with multiple ingredients
Ingredient used by multiple recipes
Recipe ingredient deletion
Recipe update synchronization

Recommendation Tests

All required ingredients matched
Partial ingredients matched
No ingredients matched
Optional ingredient missing
Duplicate selected ingredient
Inactive recipe excluded

Cart Tests

Add missing ingredient
Add same ingredient twice
Merge quantities
Different units remain separate
Unauthorized cart modification

---

71. Example Complete Recipe Data

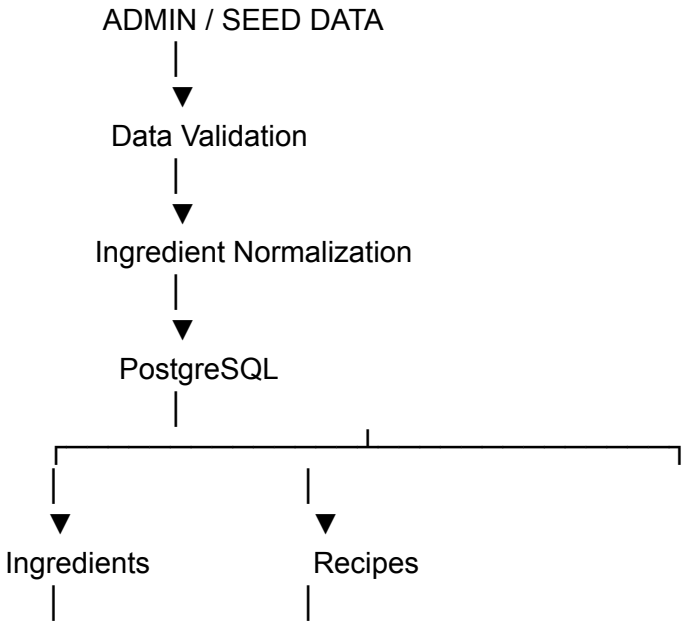

```
{
  "id": "recipe-001",
  "title": "Menemen",
  "description": "Domates, biber ve yumurta ile hazırlanan pratik bir kahvaltı.",
  "image_url": "https://example.com/images/menemen.jpg",
  "instructions": "1. Sebzeleri doğrayın. 2. Tavayı ısıtın. 3. Domatesleri ekleyin. 4. Yumurtaları ekleyin.",
  "source_url": "https://example.com/recipes/menemen",
  "youtube_url": "https://www.youtube.com/watch?v=example",
  "category": "breakfast",
  "ingredients": [
    {
      "ingredient_id": "ingredient-001",
      "name": "Domates",
      "quantity": 3,
      "unit": "adet",
      "is_optional": false
    },
```

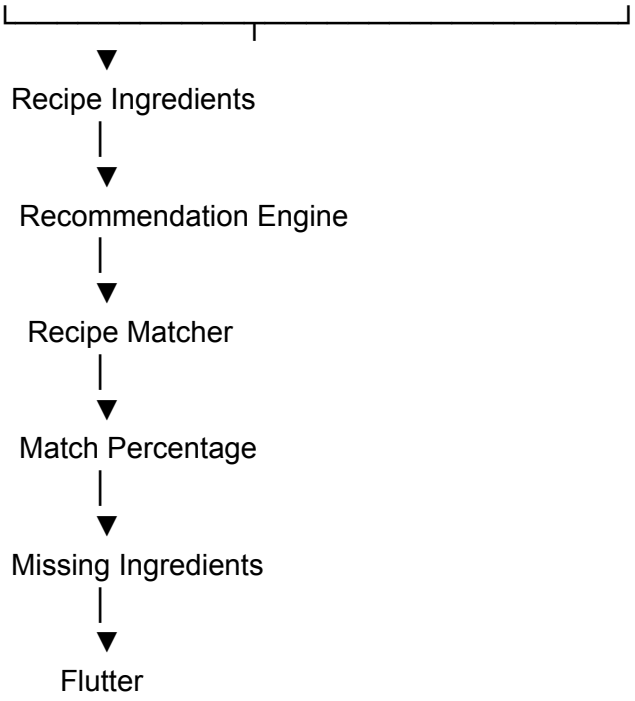
```
    {
      "ingredient_id": "ingredient-002",
      "name": "Yumurta",
      "quantity": 2,
      "unit": "adet",
      "is_optional": false
    },
    {
      "ingredient_id": "ingredient-003",
      "name": "Biber",
      "quantity": 2,
      "unit": "adet",
      "is_optional": false
    },
    {
      "ingredient_id": "ingredient-004",
      "name": "Tuz",
      "quantity": 1,
      "unit": "çay kaşığı",
      "is_optional": true
    }
  ]
}
```


---

72. Complete Data Flow

Smart Chef & Pantry AI'de ingredient ve recipe veri akışı:

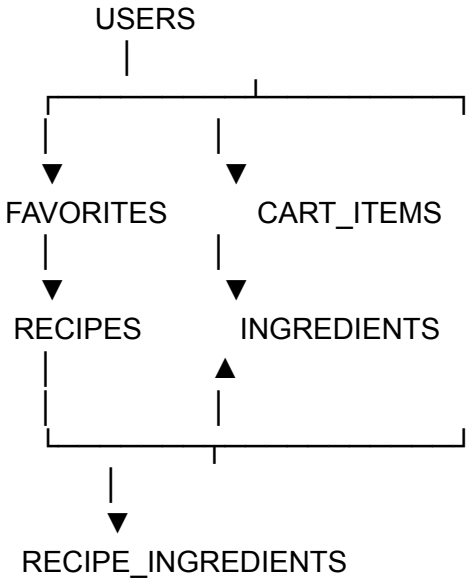




---

73. Final Data Architecture

Smart Chef & Pantry AI için temel veri ilişkisi:



Temel ilişkiler:

User 1 —— N Favorite
Recipe 1 —— N Favorite

User 1 ——— N CartItem
Ingredient 1 ——— N CartItem

Recipe 1 ——— N RecipeIngredient
Ingredient 1 ——— N RecipeIngredient

---

74. MVP Data Management Scope

MVP'de uygulanacak veri yönetimi:

✓ Ingredient CRUD
✓ Recipe CRUD
✓ Recipe–Ingredient relationship
✓ Ingredient normalization
✓ Required / Optional ingredients
✓ Quantity / Unit
✓ Recipe source URL
✓ YouTube URL
✓ Ingredient search
✓ Ingredient pagination
✓ Recipe active/inactive management
✓ Admin data management
✓ Seed data
✓ Validation
✓ Duplicate prevention
✓ Database constraints

MVP dışında bırakılan veya sonraya bırakılan özellikler:

○ Ingredient semantic AI matching
○ Automatic ingredient ontology
○ Advanced alias system
○ User-created recipes
○ Recipe reviews
○ Recipe ratings
○ Nutritional database
○ Automatic web scraping
○ Automatic recipe generation
○ AI-generated recipes
○ Personalized recipe data

---

75. Future AI Integration

Ingredient ve recipe data modeli gelecekte AI sistemlerinin kullanılmasına uygun şekilde tasarlanmıştır.

İleride:

Ingredient
↓
Embedding
↓
Semantic Similarity
↓
Ingredient Relationship

ve:

Recipe
↓
Recipe Embedding
↓
User Preference
↓
Personalized Recommendation

gibi sistemler eklenebilir.

Ancak MVP'de recommendation sonucu deterministik matching algoritması ile üretilecektir.

AI katmanı MVP'nin temel veri bütünlüğünü değiştirmemelidir.

---

76. Final Design Principles

Smart Chef & Pantry AI ingredient ve recipe veri yönetimi şu prensiplere göre geliştirilecektir:

1. Ingredient'lar global katalog olarak yönetilecektir.
2. Recipe'ler admin tarafından yönetilecektir.
3. Recipe ve ingredient arasında many-to-many ilişki kullanılacaktır.
4. `recipe_ingredients` ilişki tablosu quantity, unit ve optional bilgilerini taşıyacaktır.
5. Ingredient normalization yalnızca güvenli textual normalization yapacaktır.
6. Semantic olarak farklı ingredient'lar otomatik birleştirilmeyecektir.
7. Aynı recipe içerisinde aynı ingredient duplicate olamayacaktır.
8. Recipe'de en az bir required ingredient bulunacaktır.
9. Quantity negatif olamayacaktır.
10. Recipe ve ingredient verileri API üzerinden yönetilecektir.
11. Flutter database'e doğrudan erişmeyecektir.
12. Recipe matching backend'de gerçekleştirilecektir.
13. `match_percentage` backend tarafından hesaplanacaktır.
14. Recipe ve ingredient verileri validation'dan geçmeden aktif recommendation sistemine dahil edilmeyecektir.
15. Mümkün olduğunda fiziksel silme yerine inactive yaklaşımı kullanılacaktır.
16. Recipe creation ve relationship creation transaction içerisinde gerçekleştirilecektir.
17. Database model ile API model birbirinden ayrılacaktır.
18. Seed data kontrollü ve doğrulanmış şekilde kullanılacaktır.
19. External recipe data kullanılırken ilgili veri kullanım şartları ve lisanslar dikkate alınacaktır.
20. Veri modeli gelecekte alias, semantic matching, personalization ve AI recommendation özelliklerinin eklenmesine uygun tutulacaktır.

Bu yapı ile Smart Chef & Pantry AI'nin ingredient ve recipe veri katmanı; recommendation engine, favorites, cart ve gelecekteki AI özellikleri için tutarlı bir temel oluşturacaktır.